

Cross-Site Scripting (XSS) Scan Evidence

HIGH

Target: OWASP/bWAPP (Buggy Web Application) — Lab IT Audit

1. Informasi Pengujian

Tanggal Pengujian	24 Mei 2026
Metode	Manual testing (HTTP request injection)
Target	<code>http://localhost:8080/</code>
Modul Diuji	XSS Reflected (GET) & XSS Stored (Blog)
Security Level	Low (security_level=0)
Autentikasi	Session cookie (login: bee / bug)

2. Temuan A — Reflected XSS (GET)

Endpoint

`xss_get.php` — parameter `firstname`

Payload

```
http://localhost:8080/xss_get.php?firstname=<script>alert('XSS-bWAPP')</script>&lastname=test&form=submit
```

Bukti (response server)

Payload dipantulkan kembali ke HTML **tanpa output encoding**:

```
Welcome <script>alert('XSS-bWAPP')</script> test
```

Karena tag `<script>` tidak di-escape menjadi `<script>`, browser korban akan mengeksekusi JavaScript tersebut. Serangan dipicu lewat link berbahaya yang dikirim ke korban.

3. Temuan B — Stored XSS (Blog)

Endpoint

`xss_stored_1.php` — field `entry` (POST)

Payload

```
POST /xss_stored_1.php
entry=<script>alert('Stored-XSS-by-Audit')</script>&blog=submit&entry_add=
```

Bukti Persistence (request GET terpisah setelah penyimpanan)

Payload **tersimpan permanen di database** dan dirender mentah ke setiap pengunjung halaman blog:

```
<td><script>alert('Stored-XSS-by-Audit')</script></td>
```

Berbeda dari Reflected XSS, payload ini aktif bagi **seluruh pengguna** yang membuka halaman, tanpa perlu link khusus. Catatan: payload uji telah **dihapus kembali** setelah verifikasi untuk menjaga kebersihan lab.

4. Penilaian Risiko

Aspek	Keterangan
Severity	HIGH
OWASP Top 10 2021	A03:2021 Injection (Cross-Site Scripting)
CVSS (estimasi)	Reflected ~6.1 (Medium) · Stored ~7.4 (High)
Root Cause	Input pengguna tidak di-sanitasi saat input dan tidak di-encode saat output (output encoding)
Impact	Pencurian session cookie, session hijacking, defacement, phishing, keylogging, penyebaran malware ke pengunjung
Faktor Pemberat	Cookie session bWAPP tidak memiliki flag <code>HttpOnly</code> (lihat temuan Nikto) sehingga cookie dapat dicuri via XSS

5. Rekomendasi Remediasi

1. Terapkan **output encoding** sesuai konteks (HTML entity encoding) pada semua data yang ditampilkan, mis. `htmlspecialchars()` di PHP.
2. Lakukan **input validation** dengan whitelist pada sisi server.
3. Pasang **Content Security Policy (CSP)** untuk membatasi eksekusi script.
4. Set flag **HttpOnly** dan **Secure** pada cookie session agar tidak dapat diakses JavaScript.
5. Gunakan framework/template engine yang melakukan auto-escaping secara default.